

NODE.JS DIGITAL NOTEBOOKS

File System (`fs` Module) & JSON APIs — Student Handbook & Practical Guide

Course: Backend Web Development with Node.js

Module: File Handling & JSON Storage

Core Superpower: Giving Code Long-Term Memory

1. The Memory Problem: Variables vs. Permanent Storage

Have you ever written a program, saved information in variables, closed your computer, and found that all your data disappeared when you restarted it?

Why Variables Forget: RAM vs. Hard Drive Storage

Variables in JavaScript live inside your computer's **RAM (Random Access Memory)**. RAM is like holding a list of items in your hands or memorizing a phone number in your short-term memory—the second your program stops running or your computer goes to sleep, RAM clears out completely!

To build real-world applications (like recipe books, student managers, or digital journals), our programs need a way to write information down permanently.

2. The Digital Almira: Node.js `fs` Module

Node.js provides a powerful built-in module called **`fs` (File System)**. Think of the `fs` module as a sturdy **Digital Almira (Cupboard)** on your computer's hard drive!

CONCEPT 01 Long-Term Storage

Just like your home cupboard keeps your clothes safe even when the lights are turned off, the `fs` module saves data safely on your disk drive.

CONCEPT 02 Permanent Files

Data stored inside files using `fs` stays forever until you explicitly delete or update it.

CONCEPT 03 Built-in Superpower

You don't need to install external libraries; `fs` comes included right inside Node.js using `require('fs')`.

3. Writing & Reading Files with `fs.writeFileSync()` and `fs.readFileSync()`

Let's look at the basic functions to create, write, and read text files in Node.js:

Writing to the Almira: `fs.writeFileSync()`

This function creates a new file (or overwrites an existing one) and writes text directly into it.

```
const fs = require('fs');

// Writing a simple text note to 'notes.txt'
fs.writeFileSync('notes.txt', 'My first digital note in Node.js!');
console.log('File saved successfully in the Almirah!');
```

Reading from the Almirah: `fs.readFileSync()`

This function reads the content of a file from disk into your code. We pass `utf-8` as the encoding parameter so the raw binary data is converted into human-readable text.

```
const fs = require('fs');

// Reading 'notes.txt' back into memory
const content = fs.readFileSync('notes.txt', 'utf-8');
console.log('File Content:', content);
```

4. Unstructured vs. Structured Data: `.txt` vs. `.json`

While `.txt` files work well for simple plain sentences, complex programs need to store structured data (like objects with labels).

Format	Characteristics	Best Used For
.txt Files	Unstructured plain text. No native key-value labels. Hard for code to extract specific properties.	Simple logs, plain notes, raw text documents.
.json Files (JavaScript Object Notation)	Structured key-value pairs (`"name": "Paneer Butter Masala"`). Universal standard understood across all programming languages!	Student profiles, digital recipe cards, e-commerce products, configuration settings.

The Digital Recipe Card Metaphor

A `.txt` file is like a handwritten loose napkin note: *"Paneer 200g butter tomatoes spices cook 20 mins"*. A `.json` file is a neatly organized printed recipe card with labeled fields: `{"title": "Paneer Butter Masala", "prepTime": "20 mins", "ingredients": ["Paneer", "Butter", "Tomatoes"]}`. Computers love organized labels!

5. Packing & Unpacking Data: `JSON.stringify()` & `JSON.parse()`

Files on disk can only store text strings. To save JavaScript objects, we must ****Pack**** them into JSON strings before saving, and ****Unpack**** them back into JavaScript objects after reading!

Pack (Stringify)

``JSON.stringify(object)``

Converts a live JavaScript Object into a formatted JSON String so it can be written to disk with ``fs.writeFileSync()``.

Unpack (Parse)

``JSON.parse(jsonString)``

Converts a JSON String read from disk back into a fully functional JavaScript Object so you can access properties like ``recipe.title``!

Complete Code Workflow: Building a Recipe Saver

```
const fs = require('fs');

// 1. Create a JavaScript Object
const recipe = {
  title: "Special Masala Chai",
  prepTime: "10 mins",
  rating: 5
};

// 2. PACK: Convert Object -> JSON String
const recipeData = JSON.stringify(recipe, null, 2);

// 3. SAVE: Write to 'recipe.json'
fs.writeFileSync('recipe.json', recipeData);

// 4. READ & UNPACK: Read from disk and convert JSON String -> Object
const rawData = fs.readFileSync('recipe.json', 'utf-8');
const savedRecipe = JSON.parse(rawData);

console.log("Recipe Title:", savedRecipe.title); // Output: Special Masala Chai
```

6. Key Takeaways for Node.js Developers

- **Permanent Memory:** Node.js ``fs`` module provides your programs with long-term memory that survives computer restarts.
- **Structured Standard:** JSON is the universal standard for exchanging and storing structured data across modern web applications.
- **Two-Step Cycle:** Always ``JSON.stringify()`` before writing objects to disk, and ``JSON.parse()`` after reading strings from disk!

7. Extended Worksheet: Build Your Digital Notebook Application

Design your own custom Node.js Digital Notebook application! Complete all parts of this blueprint.

Part A: Application Concept Profile

Application Title: _____

Primary Purpose: (e.g., Student Record Keeper, Game High Scores, Personal Expense Tracker)

File Name to Create: _____ (e.g., data.json)

Part B: Design Your Data Object Structure

Define a JavaScript Object structure with 4 labeled properties for your application:

```
const myDataObject = {  
  property1: _____,  
  property2: _____,  
  property3: _____,  
  property4: _____  
};
```

Part C: Code Implementation Plan

1. Write the Node.js code line to import the File System module:

2. Write the code line to PACK (stringify) your object into JSON format:

3. Write the code line to SAVE the packed string to your file synchronously:

4. Write the code lines to READ the file back and UNPACK (parse) it into a variable:

SECTION 8: STUDENT PRACTICE QUESTIONS

Node.js `fs` & JSON Knowledge Check — Answer all questions in the space provided.

Instructions: Test your understanding of Node.js file system methods, synchronous operations, JSON formats, stringify/parse workflows, and data persistence. Write your answers clearly in the spaces below. (No answers included).

Q1. The Memory Problem & RAM vs. Disk

Explain why JavaScript variables lose their data when a Node.js program stops running. Compare the role of computer RAM with the long-term storage provided by the `fs` module (Digital Almirah).

Q2. File System Method Breakdown

Identify the purpose of `fs.writeFileSync()` and `fs.readFileSync()`. Why is it necessary to include the `utf-8` parameter when calling `fs.readFileSync()`?

Q3. Unstructured `.txt` vs. Structured `.json`

Contrast plain text (`.txt`) files with JSON (`.json`) files using the *Digital Recipe Card* metaphor. Why do backend software developers prefer JSON when storing complex application data?

Q4. Packing & Unpacking Data

What specific roles do `JSON.stringify()` and `JSON.parse()` perform in file operations? Explain what would happen if you attempted to write a raw JavaScript Object directly into `fs.writeFileSync()` without stringifying it first.

Q5. Code Debugging & Syntax Analysis

Examine the following buggy Node.js snippet and identify **two errors** that will prevent it from reading and printing the student's score correctly:

```
const fs = require('fs');
const data = fs.readFileSync('student.json');
console.log(data.score);
```

Q6. Application Workflow Challenge

Write out a step-by-step logic plan (pseudocode or JavaScript) for an application that reads a list of recipes from `recipes.json`, adds a new recipe object to the list, and saves the updated list back to `recipes.json`.